

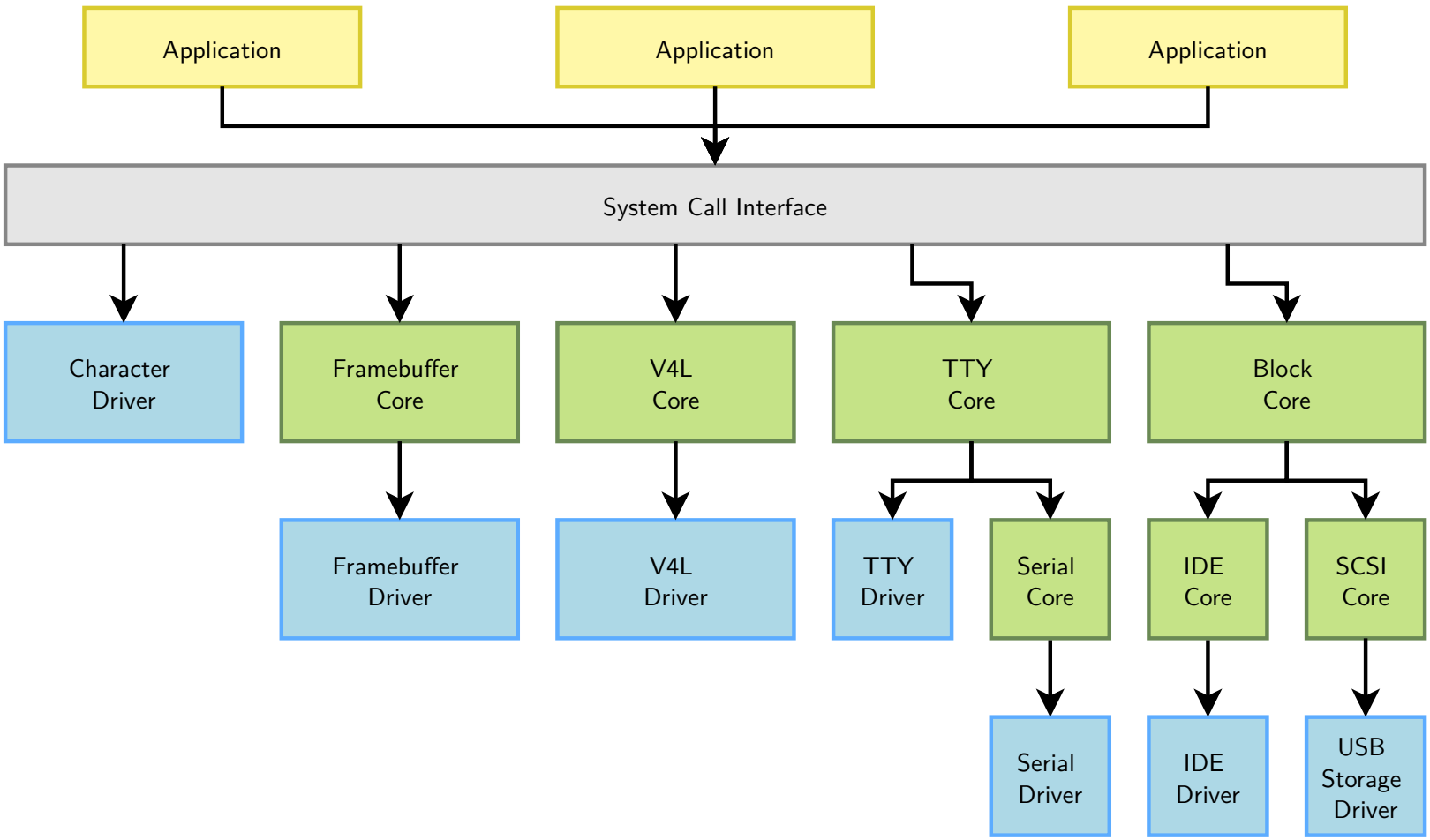
The concept of kernel frameworks



Beyond character drivers: kernel frameworks

- Many device drivers are not implemented directly as character drivers
- They are implemented under a *framework*, specific to a given device type (framebuffer, V4L, serial, etc.)
  - The driver plugs into a framework API rather than into the generic struct file_operations
  - The framework implements once struct file_operations to expose character devices to userspace
  - That implementation is shared across all framework devices: it minimizes driver boilerplate and it provides a coherent userspace interface whatever driver is being used

Example: Some Kernel Frameworks





Example: the input subsystem

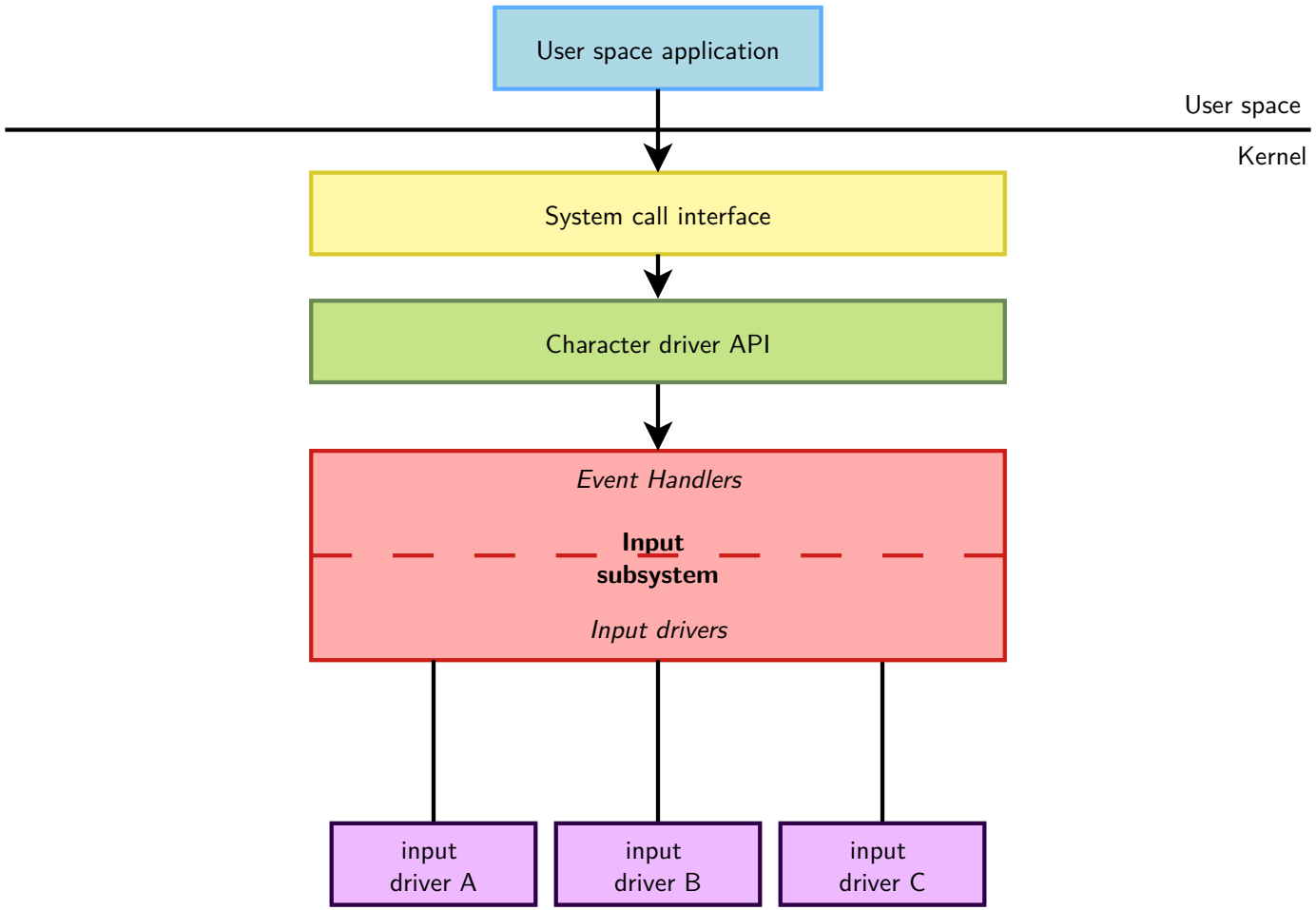


What is the input subsystem?

- The input subsystem takes care of all the input events coming from the human user.
- Initially written to support the USB *HID* (Human Interface Device) devices, it quickly grew up to handle all kinds of inputs (using USB or not): keyboards, mice, joysticks, touchscreens, etc.
- The input subsystem is split in two parts:
  - **Device drivers**: they talk to the hardware (for example via USB), and provide events (keystrokes, mouse movements, touchscreen coordinates) to the input core
  - **Event handlers**: they get events from drivers and pass them where needed via various interfaces (most of the time through `evdev`)
- In user space it is usually used by the graphic stack such as *X.Org*, *Wayland* or *Android's InputManager*.



Input subsystem diagram





Input subsystem overview

- Kernel option CONFIG_INPUT
  - `menuconfig INPUT`
    - `tristate "Generic input layer (needed for keyboard, mouse, ...)"`
- Implemented in drivers/input/
  - input.c, input-poller.c, evdev.c...
- Defines the user/kernel API
  - include/uapi/linux/input.h
- Defines the set of operations an input driver must implement and helper functions for the drivers
  - struct input_dev for the device driver part
  - struct input_handler for the event handler part
  - include/linux/input.h



Input subsystem API 1/3

An *input device* is described by a very long struct input_dev structure, an excerpt is:

```
struct input_dev {
    const char *name;
    [...]
    struct input_id id;
    [...]
    unsigned long evbit[BITS_TO_LONGS(EV_CNT)];
    unsigned long keybit[BITS_TO_LONGS(KEY_CNT)];
    [...]
    int (*getkeycode)(struct input_dev *dev,
                      struct input_keymap_entry *ke);
    [...]
    int (*open)(struct input_dev *dev);
    [...]
    int (*event)(struct input_dev *dev, unsigned int type,
                 unsigned int code, int value);
    [...]
};
```

Before being used, this structure must be allocated and initialized, typically with:

```
struct input_dev *devm_input_allocate_device(struct device *dev);
```



Input subsystem API 2/3

- Depending on the type of events that will be generated, the input bit fields `evbit` and `keybit` must be configured: For example, for a button we only generate EV_KEY type events, and from these only BTN_0 events code:

```
set_bit(EV_KEY, myinput_dev.evbit);
set_bit(BTN_0, myinput_dev.keybit);
```

- Once the *input device* is allocated and filled, the function to register it is:
  `int input_register_device(struct input_dev *);`



Input subsystem API 3/3

The events are sent by the driver to the event handler using

```
void input_event(struct input_dev *dev, unsigned int type, unsigned int code, int value)
```

- The event types are documented in input/event-codes
- An event is composed by one or several input data changes (packet of input data changes) such as the button state, the relative or absolute position along an axis, etc..
- The input subsystem provides other wrappers such as:
  - input_report_key()
  - input_report_abs()

After submitting potentially multiple events, the *input* core must be notified by calling:

```
void input_sync(struct input_dev *dev)
```



Example from drivers/hid/usbhid/usbmouse.c

```
static void usb_mouse_irq(struct urb *urb)
{
        struct usb_mouse *mouse = urb->context;
        signed char *data = mouse->data;
        struct input_dev *dev = mouse->dev;
        ...

        input_report_key(dev, BTN_LEFT,   data[0] & 0x01);
        input_report_key(dev, BTN_RIGHT,  data[0] & 0x02);
        input_report_key(dev, BTN_MIDDLE, data[0] & 0x04);
        input_report_key(dev, BTN_SIDE,   data[0] & 0x08);
        input_report_key(dev, BTN_EXTRA,  data[0] & 0x10);

        input_report_rel(dev, REL_X,     data[1]);
        input_report_rel(dev, REL_Y,     data[2]);
        input_report_rel(dev, REL_WHEEL, data[3]);

        input_sync(dev);
        ...
}
```



Polling input devices

- The input subsystem provides an API to support simple input devices that *do not raise interrupts* but have to be *periodically scanned or polled* to detect changes in their state.
- Setting up polling is done using input_setup_polling():

```
int input_setup_polling(struct input_dev *dev, void (*poll_fn)(struct input_dev *dev));
```

- `poll_fn` is the function that will be called periodically.
- The polling interval can be set using input_set_poll_interval() or input_set_min_poll_interval() and input_set_max_poll_interval()



evdev user space interface

- The main user space interface to *input devices* is the **event interface**
- Each *input device* is represented as a `/dev/input/event<X>` character device
- A user space application can use blocking and non-blocking reads, but also `select()` (to get notified of events) after opening this device.
- Each read will return struct input_event structures of the following format:

```
struct input_event {
        struct timeval time;
        unsigned short type;
        unsigned short code;
        unsigned int value;
};
```

- A very useful application for *input device* testing is `evtest`, from https://cgit.freedesktop.org/evtest/



Device-managed allocations



Device managed allocations

- The `probe()` function is typically responsible for allocating a significant number of resources: memory, mapping I/O registers, registering interrupt handlers, etc.
- These resource allocations have to be properly freed:
  - In the `probe()` function, in case of failure
  - In the `remove()` function
- This required a lot of failure handling code that was rarely tested
- To solve this problem, *device managed* allocations have been introduced.
- The idea is to associate resource allocation with the `struct device`, and automatically release those resources
  - When the device disappears
  - When the device is unbound from the driver
- Functions prefixed by `devm_`
- See driver-api/driver-model/devres for details



Device managed allocations: memory allocation example

- Normally done with `kmalloc(size_t, gfp_t)`, released with `kfree(void *)`
- Device managed with `devm_kmalloc(struct device *, size_t, gfp_t)`

Without devm functions

```
int foo_probe(struct platform_device *pdev)
{
        struct foo_t *foo = kmalloc(sizeof(struct foo_t),
                                    GFP_KERNEL);
        /* Register to framework, store
         * reference to framework structure in foo */
        ...
        if (failure) {
                kfree(foo);
                return -EBUSY;
        }
        platform_set_drvdata(pdev, foo);
        return 0;
}

void foo_remove(struct platform_device *pdev)
{
        struct foo_t *foo = platform_get_drvdata(pdev);
        /* Retrieve framework structure from foo
           and unregister it */
        ...
        kfree(foo);
}
```

With devm functions

```
int foo_probe(struct platform_device *pdev)
{
        struct foo_t *foo = devm_kmalloc(&pdev->dev,
                                         sizeof(struct foo_t),
                                         GFP_KERNEL);
        /* Register to framework, store
         * reference to framework structure in foo */
        ...
        if (failure)
                return -EBUSY;
        platform_set_drvdata(pdev, foo);
        return 0;
}

void foo_remove(struct platform_device *pdev)
{
        struct foo_t *foo = platform_get_drvdata(pdev);
        /* Retrieve framework structure from foo
           and unregister it */
        ...
        /* foo automatically freed */
}
```



Device managed allocations caveats

- Cleanup is done when the `struct device` is cleaned up. There is no reference counting or anything like that.
- Don't use if the allocated memory is used outside of the device node. E.g. if the userspace device file is still open after remove.
- Be very careful when there are circular references.
- "Why is `devm_kzalloc()` harmful and what can we do about it", Laurent Pinchart, LPC 2022



Driver data structures and links



Driver data layout Three main data structures:

- Bus-specific device structure (struct i2c_client, struct usb_dev, etc)
  - It *always embeds* a struct device
  - A pointer to it is passed to the `probe()` function
- Framework-specific device struct (struct input_dev, struct rtc_device, etc)
  - It *might embed* a struct device if the framework wants a sysfs device
  - Careful! We might have one bus struct device and one framework struct device for the same device!
- Driver private data
  - The structure is driver specific, with space for all device state information
  - It stores references to both the bus and framework devices



Driver data allocation strategies

Private data embeds the framework device therefore a single allocation allocates both the framework device and the private data.

```
struct imx_port {
        struct uart_port port;
        struct timer_list timer;
        unsigned int old_status;
        int txirq, rxirq, rtsirq;
        [...]
};

sport = devm_kzalloc(&pdev->dev,
                     sizeof(*sport),
                     GFP_KERNEL);
if (!sport)
        return -ENOMEM;
```

The framework exposes an helper to allocate the framework device, with space at the end to put the private data.

```
struct da311_data *data;
struct iio_dev *idev;

idev = devm_iio_device_alloc(
                     &client->dev,
                     sizeof(*data));
if (!idev)
        return -ENOMEM;

data = iio_priv(idev);
data->client = client;
```

The framework device and private data are allocated separately.

```
struct rtc_device *rtc;
struct ds1305 *ds1305;

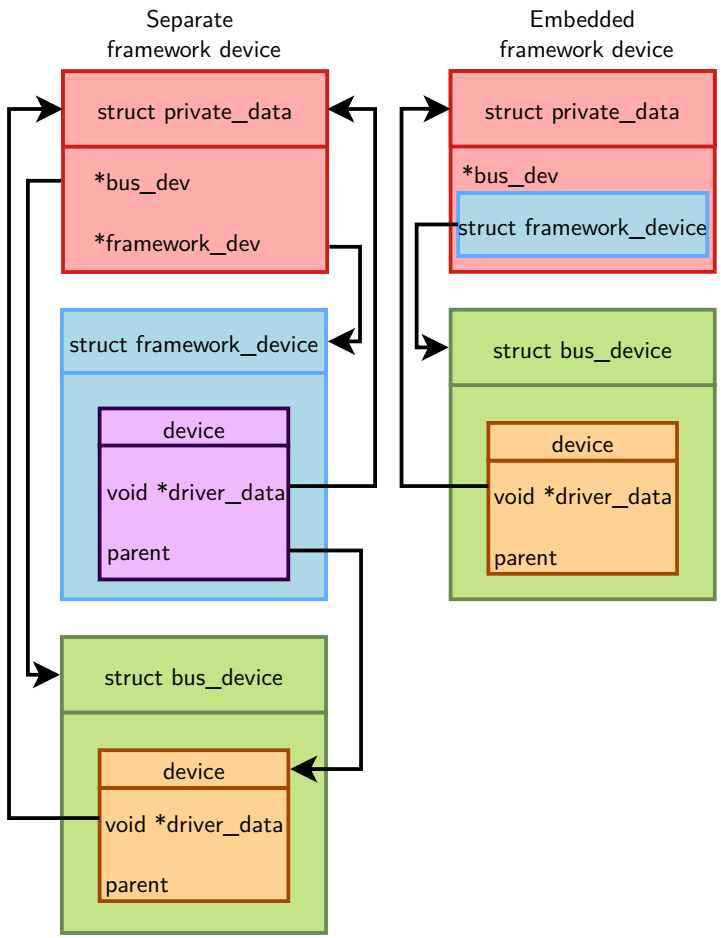
ds1305 = devm_kzalloc(&spi->dev,
                      sizeof(*ds1305),
                      GFP_KERNEL);
if (!ds1305)
        return -ENOMEM;

rtc = devm_rtc_allocate_device(...);
if (IS_ERR(rtc))
        return PTR_ERR(rtc);
```



Links between data structures

- Inside bus callbacks, we get passed our bus device
  - Inside struct device, a pointer-sized field `dev->driver_data` is reserved for driver usage
  - Use dev_set_drvdata() at `probe()` time to put a reference to our private data
  - From bus callbacks, we can retrieve our private data using dev_get_drvdata()
- Inside framework callbacks, we get passed our framework device
  - If our framework device is embedded in our private data, we use container_of() that works using compiler provided `offsetof()`
  - Otherwise, we use the framework device `dev->driver_data` and retrieve our private data reference



Practical lab - Expose the Nunchuk to user space



- Extend the Nunchuk driver to expose the Nunchuk features to user space applications, as an *input* device.
- Test the operation of the Nunchuk using `evtest`